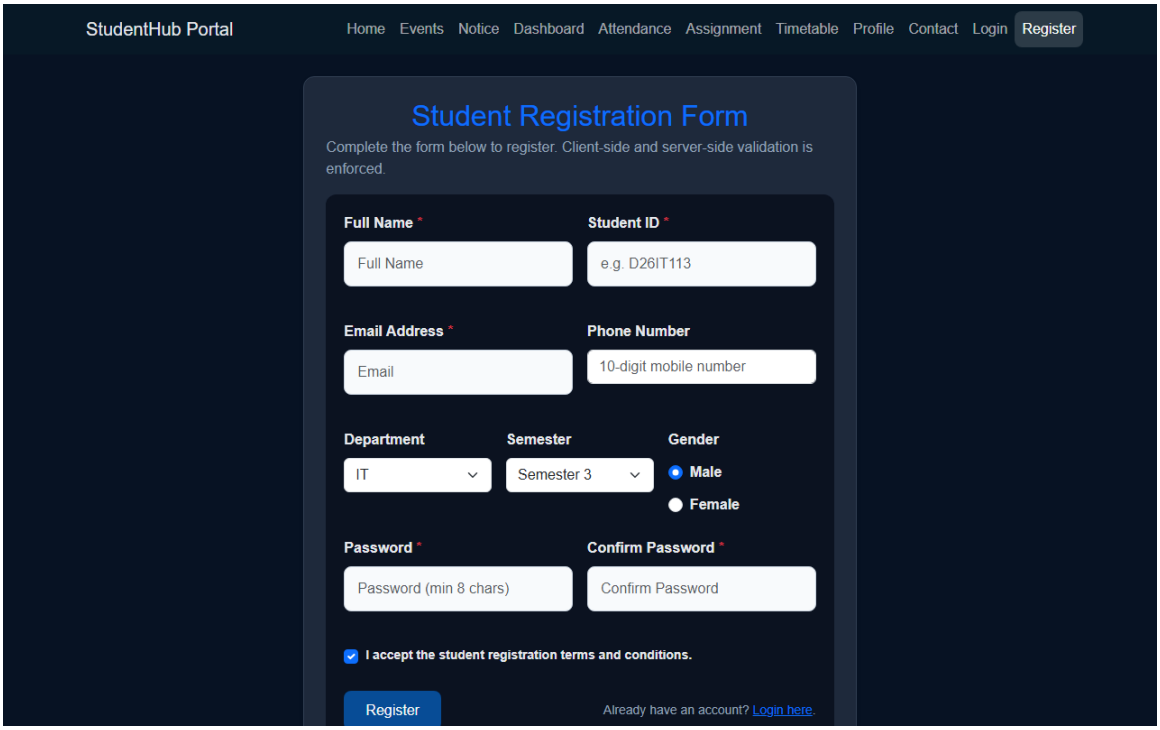
PRACTICAL – 5

Problem Definition: Create a student registration form with HTML5 input types and JavaScript validation for name, email, mobile number, password, confirm password, course, year, gender, and terms acceptance. Use Regular Expression for validations.

Output:

Student Registration Form with Frontend Validation:



Question & Answer:

1. Are correct input types and attributes used?

Ans. Yes. The form uses appropriate HTML5 input types such as type="text", type="email", type="tel", type="password", type="radio", and type="checkbox". Useful HTML5 validation attributes such as required, pattern, minlength, and autocomplete are also applied to ensure proper baseline validation.

2. Are validation errors displayed near the relevant fields?

Ans. Yes. Validation error messages are rendered dynamically inside dedicated small text containers located immediately below each corresponding input field. Input fields receive distinct visual feedback (field-invalid or field-valid classes) to clearly highlight errors to the user in real time.

3. Is password strength checked?

Ans. Yes. The password field is validated to ensure a minimum length of 8 characters containing a mixture of letters and numbers. Additionally, the confirm password field is

validated in real time to guarantee that both entered passwords match before submission is permitted.

4. Is the form accessible using keyboard and screen-reader-friendly labels?

Ans. Yes. All form controls are explicitly associated with descriptive <label> elements using matching id and for attributes. Error messages use aria-live="polite" and inputs toggle aria-invalid="true" to announce errors to screen readers, while keyboard tab navigation remains smooth and intuitive.

Conclusion / Learning Outcome:

This practical provided hands-on experience in engineering robust, accessible, and user-friendly registration forms. Client-side regular expression pattern testing was applied to student names, university emails, mobile numbers, and passwords. Immediate feedback mechanisms were designed to improve form completion rates while preventing malformed payloads from reaching downstream server systems.

Github link :- https://github.com/07vish/sh.git